

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

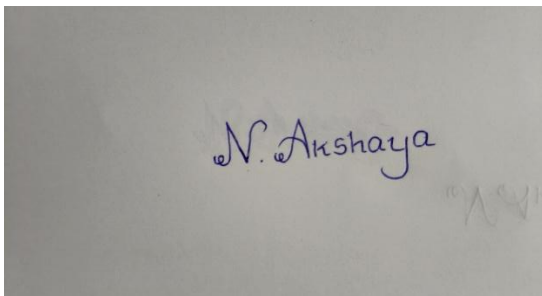
Duration: 30 minutes

Marks: 25

Code of Conduct:

I **N.AKSHAYA(192425129)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

A photograph of a piece of paper with a handwritten signature in blue ink. The signature reads "N. Akshaya". There is some faint, illegible handwriting below the main signature.

Signature of the student:

Name of the student: N.AKSHAYA

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

A **learning rate** determines the size of the steps taken by the gradient descent algorithm while updating model parameters to minimize the loss function. Choosing a **high initial learning rate followed by gradual decay** is a widely used optimization strategy.

Effect on Convergence

- **High Initial Learning Rate**
 - Takes large steps toward the minimum.
 - Speeds up the early stages of training.
 - Helps the optimizer move quickly through regions where the loss decreases slowly.
 - Can escape shallow local irregularities or flat regions more easily.
- **Gradually Decreasing Learning Rate**
 - Reduces the update size as training progresses.
 - Allows fine adjustments near the optimum.
 - Improves convergence stability.
 - Produces a more accurate final solution with less oscillation.

Why This Strategy Can Outperform a Constant Learning Rate

A decaying learning rate performs better than a constant learning rate when:

1. The loss function is **convex** and smooth.
2. The initial learning rate is **large enough** to accelerate learning but **not so large** that it causes divergence.
3. The decay schedule is properly chosen (step decay, exponential decay, cosine decay, etc.).
4. The optimization requires both **fast exploration initially** and **precise convergence later**.

Compared with a constant learning rate:

Constant Learning Rate	High Initial Rate + Decay
Slower initial convergence	Faster initial convergence
May oscillate near optimum	Stable convergence near optimum
May require more iterations	Usually fewer iterations
Difficult to balance speed and accuracy	Provides both speed and precision

Possible Risks

Despite its advantages, this strategy also has some risks:

- **Learning rate too high initially**
 - Parameters may overshoot the minimum.
 - Loss may oscillate or even diverge.
 - Training can become unstable.
- **Learning rate decays too quickly**
 - Updates become extremely small.
 - Model may stop learning before reaching the optimal solution.
- **Learning rate decays too slowly**
 - Oscillations continue near the optimum.
 - Final convergence becomes slower.

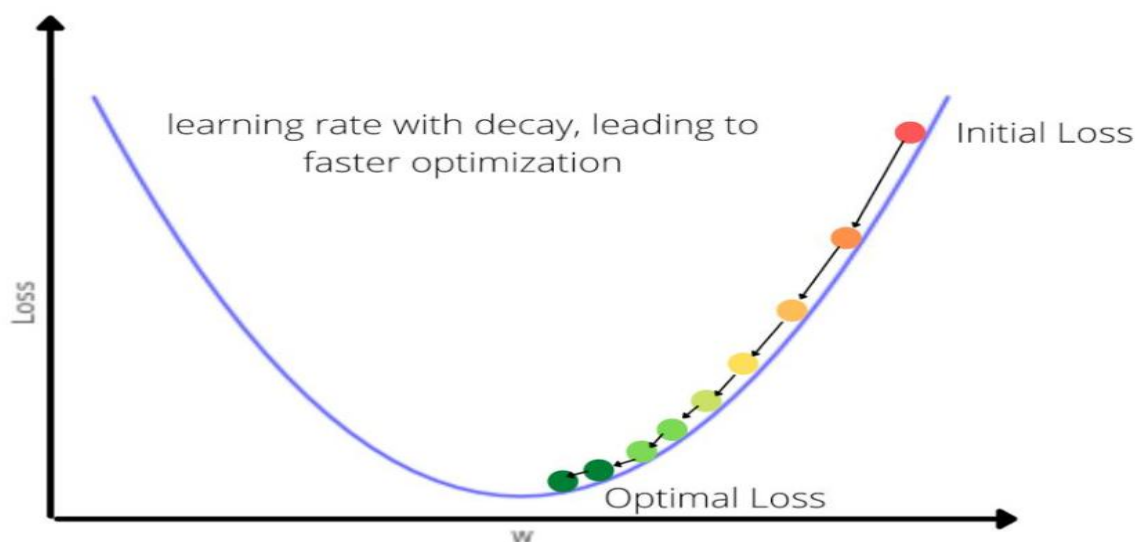
Justification

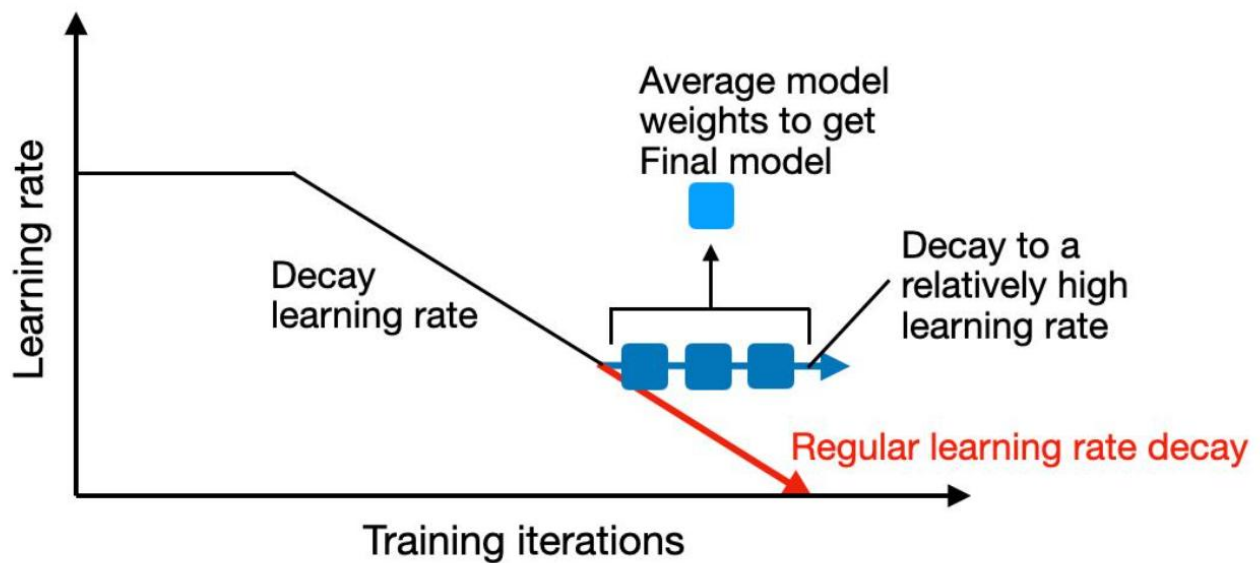
A high learning rate enables rapid progress during the early training stage when the parameters are far from the optimum. As the optimizer approaches the minimum, reducing the learning rate allows smaller and more precise updates, preventing overshooting and improving stability. Therefore, **a high initial learning rate followed by gradual decay combines fast optimization with accurate convergence**, making it more effective than a constant learning rate in many convex optimization problems when the learning rate schedule is properly selected.

Conclusion

Using a **high initial learning rate with gradual decay** is an effective optimization strategy because it:

- Accelerates early learning,
- Reduces training time,
- Improves convergence accuracy,
- Provides greater stability near the optimum.





2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer:

Suppose the observations are

$$x_1, x_2, \dots, x_n \sim N(\mu, \sigma^2)$$

where:

- **Unknown parameter:** Mean μ
- **Known parameter:** Variance σ^2

Step 1: Likelihood Function

The probability density function (PDF) of a Gaussian distribution is

$$f(x_i|\mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

For **n independent observations**, the likelihood function is

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Step 2: Log-Likelihood Function

Taking the natural logarithm,

$$\ell(\mu) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Step 3: Differentiate with Respect to μ

Differentiate the log-likelihood:

$$\frac{d\ell(\mu)}{d\mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu)$$

Set the derivative equal to zero:

$$\sum_{i=1}^n (x_i - \mu) = 0$$

$$\sum_{i=1}^n x_i = n\mu$$

Hence,

Maximum Likelihood Estimator

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

Since

$$\hat{\mu} = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

every observation contributes equally to the estimate.

If an Outlier is Present

- A very large outlier increases the sample mean.
- A very small outlier decreases the sample mean.
- Even a single extreme value can significantly change the estimate.

Example

Without outlier:

$$10, 12, 11, 13, 14$$

$$\hat{\mu} = 12$$

With an outlier:

$$10, 12, 11, 13, 100$$
$$\hat{\mu} = 29.2$$

The estimated mean shifts far from the true center of the data.

Observation:

MLE is **highly sensitive to outliers** because it minimizes the squared error, giving extreme observations a large influence.

Effect of Increasing Sample Size

When the number of observations **n increases**:

- The sample mean becomes closer to the true population mean.
- Random sampling error decreases.
- The variance of the estimator becomes

$$Var(\hat{\mu}) = \frac{\sigma^2}{n}$$

As **n increases**, this variance decreases.

Therefore

- The estimate becomes more stable.
- Accuracy improves.
- Confidence in the estimate increases.
- The estimator converges to the true mean (Law of Large Numbers).

Implications for the Robustness of MLE

Situation	Effect on MLE
Presence of outliers	Estimate changes significantly
Larger sample size	Estimate becomes more accurate
Small sample size	More sensitive to extreme values
Large sample size with few outliers	Outlier influence is reduced but not eliminated

Discussion

- **Strengths**
 - Efficient estimator for Gaussian data.
 - Unbiased estimator of the mean.
 - Consistent estimator (approaches the true parameter as sample size increases).
 - Easy to compute.
- **Weaknesses**
 - Not robust to outliers.
 - Performance deteriorates when data violate the Gaussian assumption.
 - Squared-error loss gives excessive weight to extreme observations.

Conclusion:

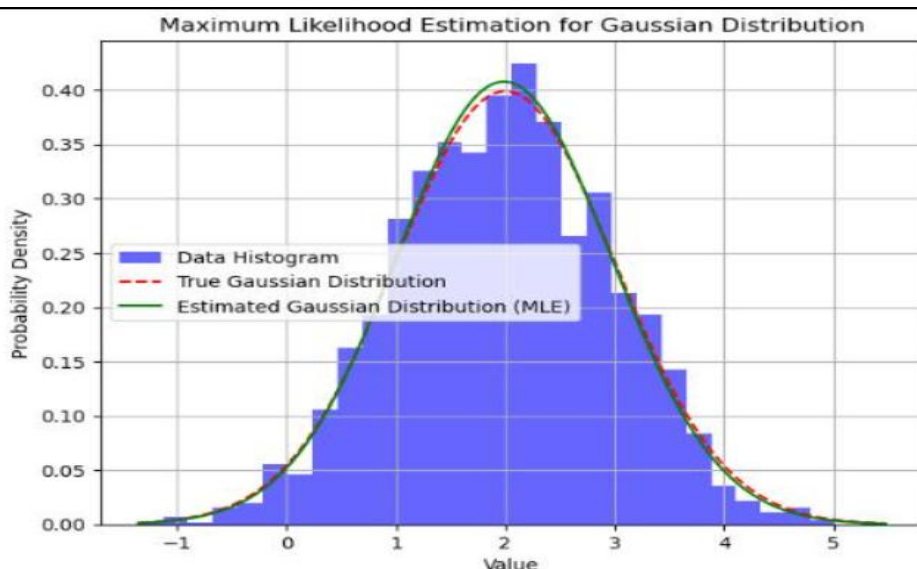
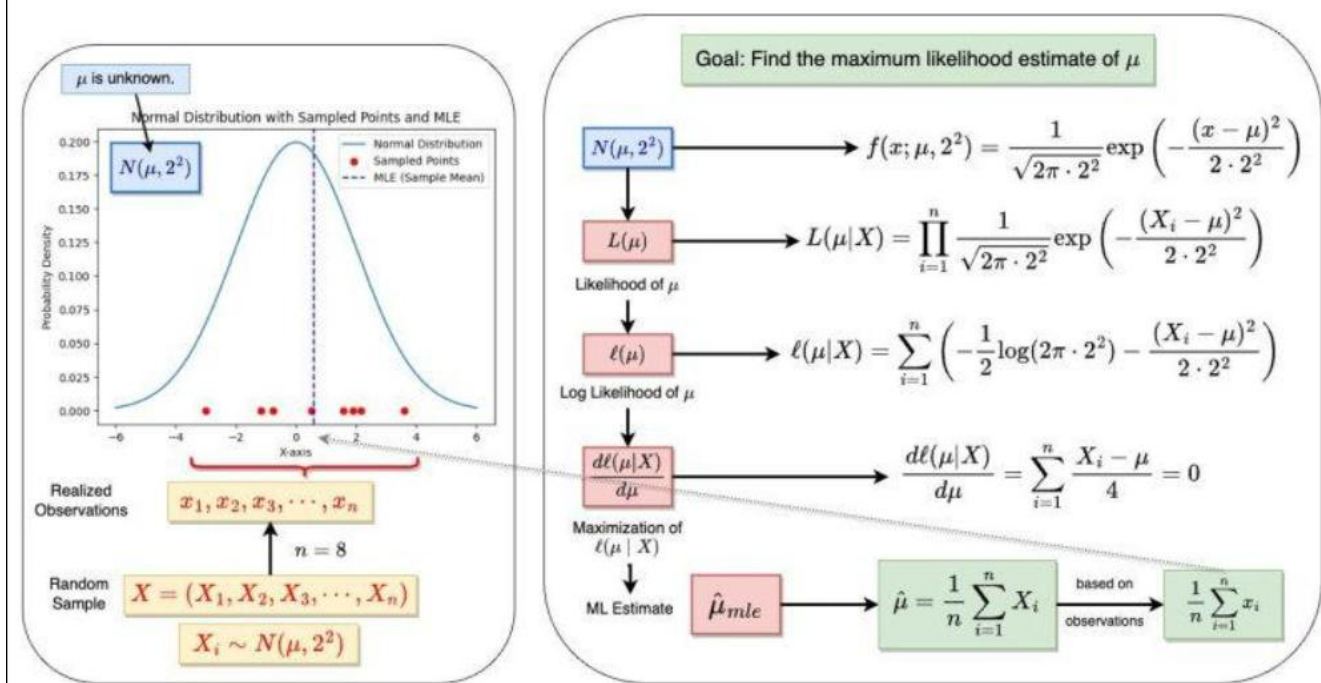
For a Gaussian distribution with known variance,

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

is the Maximum Likelihood Estimator of the mean.

- **With outliers:** The estimate shifts toward the extreme values, showing that MLE is **not robust**.
- **As sample size increases:** The estimator becomes more accurate and stable because its variance decreases as σ^2/n .

Maximum Likelihood Estimation (MLE)



3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Problem Statement

The dataset contains:

- **Features (p):** 10,000
- **Training samples (n):** 500

Since $p \gg n$ ($10,000 > 500$), this is known as a **high-dimensional, low-sample-size (HDLSS)** problem. Such datasets are prone to **overfitting**, where the model memorizes the training data instead of learning patterns that generalize to unseen data.

Challenges in Training the Model

1. Curse of Dimensionality

- The feature space becomes extremely large.
- Data points become sparse.
- Learning meaningful patterns becomes difficult.

2. Overfitting

- The model may fit noise instead of useful information.
- High training accuracy but poor test accuracy.

3. Computational Cost

- Training with 10,000 features requires more memory and computation time.

4. Irrelevant and Redundant Features

- Many features may not contribute to prediction.
- Correlated features increase model complexity.

5. Poor Generalization

- The model performs well on training data but fails on new data.

Proposed Machine Learning Pipeline

Input Dataset



Data Cleaning



Feature Scaling



Feature Selection



Dimensionality Reduction (PCA)



Train-Test Split / Cross Validation



Regularized Classifier

(L1/L2 Logistic Regression or SVM)



Hyperparameter Tuning



Model Evaluation

Step 1: Data Preprocessing

Purpose

- Remove missing values.
- Remove duplicate samples.
- Normalize numerical features.

Justification

Scaling ensures all features contribute equally and improves optimization for models such as Logistic Regression and SVM.

Step 2: Feature Selection

Methods

- Variance Threshold
- Chi-Square Test
- Mutual Information
- Recursive Feature Elimination (RFE)
- LASSO (L1 Regularization)

Justification

Feature selection removes:

- Irrelevant features
- Noisy features
- Redundant features

Example:

Original Features = 10,000

↓

Selected Features = 500

Benefits:

- Faster training
- Lower memory usage
- Better prediction accuracy
- Reduced overfitting

Step 3: Dimensionality Reduction

Method

Principal Component Analysis (PCA)

Example:

500 selected features

↓

100 principal components

Justification

PCA:

- Removes feature correlation.
- Preserves most of the variance.
- Produces compact feature representations.
- Simplifies the learning problem.

Step 4: Regularized Classification Model

Suitable algorithms include:

- Logistic Regression (L1/L2 Regularization)
- Linear SVM
- Ridge Classifier

Why Regularization?

Regularization adds a penalty to the loss function:

- Prevents excessively large model weights.
- Encourages simpler models.
- Reduces model variance.

L1 Regularization (LASSO):

- Drives some coefficients to zero.
- Performs feature selection automatically.

L2 Regularization (Ridge):

- Shrinks coefficients.
- Improves stability.

Step 5: Cross-Validation

Use **k-Fold Cross Validation** (e.g., $k = 5$ or 10).

Benefits

- Uses all training samples efficiently.
- Produces a reliable estimate of model performance.
- Helps detect overfitting.

Step 6: Hyperparameter Tuning

Optimize parameters such as:

- Regularization strength (λ or C)
- Number of PCA components
- Number of selected features

Methods:

- Grid Search
- Random Search

Step 7: Model Evaluation

Evaluate using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Compare training and validation performance to ensure the model generalizes well.

How This Pipeline Mitigates Overfitting

Step	How It Reduces Overfitting
Data Cleaning	Removes noisy or inconsistent data
Feature Selection	Eliminates irrelevant features
PCA	Reduces dimensionality while retaining important information
Regularization	Penalizes overly complex models and limits large weights
Cross-Validation	Detects poor generalization during training
Hyperparameter Tuning	Finds model complexity that balances bias and variance

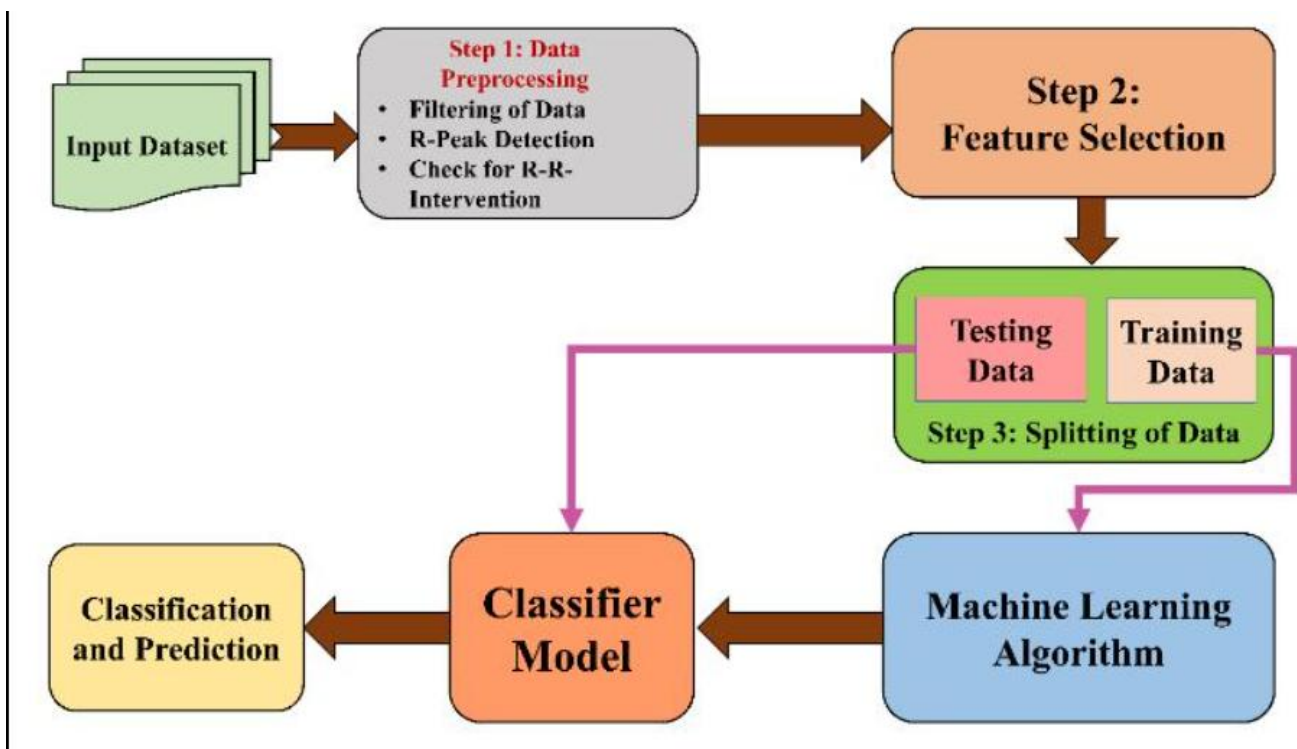
Advantages of the Proposed Pipeline

- Handles **high-dimensional data** efficiently.
- Reduces computational cost.
- Removes redundant and noisy features.
- Improves model interpretability.
- Prevents overfitting.
- Produces better performance on unseen data.

Conclusion

For a dataset with **10,000 features and only 500 training samples**, the main challenge is the **curse of dimensionality** and the high risk of **overfitting**. An effective pipeline consists of:

1. **Data preprocessing and feature scaling**
2. **Feature selection** to remove irrelevant variables
3. **PCA** for dimensionality reduction
4. **Regularized classifiers** (e.g., L1/L2 Logistic Regression or Linear SVM)
5. **Cross-validation and hyperparameter tuning**



4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

Introduction

A **Multilayer Perceptron (MLP)** is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Each hidden neuron applies a **sigmoid activation function**, allowing the network to learn complex nonlinear relationships between inputs and outputs.

The sigmoid activation function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

where:

- x = weighted sum of inputs
- $\sigma(x)$ = output of the neuron (between 0 and 1)

Why Adding More Hidden Neurons Increases Representational Power

Each hidden neuron learns a different feature or pattern from the input data.

For an MLP with one hidden layer, the output is:

$$y = \sum_{j=1}^m v_j \sigma \left(\sum_{i=1}^n w_{ij} x_i + b_j \right) + c$$

where:

- m = number of hidden neurons
- n = number of input features
- w_{ij} = weights from input to hidden layer
- v_j = weights from hidden to output layer
- b_j = hidden-layer bias
- c = output bias

Effect of Increasing Hidden Neurons

Suppose:

- 5 hidden neurons → learns 5 nonlinear features.
- 20 hidden neurons → learns 20 nonlinear features.
- 100 hidden neurons → learns many more complex patterns.

Thus, increasing the number of hidden neurons:

- Increases the number of trainable parameters.
- Enables learning of more complex nonlinear decision boundaries.
- Improves the ability to approximate complicated functions.

According to the **Universal Approximation Theorem**, an MLP with a sufficient number of hidden neurons can approximate any continuous function to arbitrary accuracy, provided enough data and appropriate weights.

Advantages of More Hidden Neurons

- Learns richer feature representations.
- Models highly nonlinear relationships.
- Improves flexibility and expressive power.
- Can achieve lower training error.

Why More Hidden Neurons Do Not Always Improve Performance

Although more hidden neurons increase the network's capacity, they can also introduce several problems.

1. Overfitting

Overfitting occurs when the network memorizes the training data instead of learning general patterns.

Characteristics:

- Very high training accuracy.
- Poor accuracy on unseen (test) data.
- Learns noise as if it were meaningful information.

For example:

- **Small network:** 95% training accuracy, 93% testing accuracy.

- **Very large network:** 100% training accuracy, 80% testing accuracy.

The larger network fits the training data perfectly but generalizes poorly.

2. Poor Generalization

Generalization is the ability of a model to perform well on new, unseen data.

When the network becomes excessively complex:

- It learns unnecessary details.
- Prediction performance on unseen data decreases.
- Model variance increases.

3. Increased Computational Cost

More hidden neurons result in:

- More weights and biases.
- Longer training time.
- Greater memory requirements.
- Higher computational complexity.

4. Risk of Vanishing Gradients

With sigmoid activation:

- Large positive or negative inputs cause the sigmoid to saturate.
- Gradients become very small.
- Weight updates slow down, making training difficult, especially in larger networks.

Overfitting vs. Generalization

Overfitting	Generalization
Memorizes training data	Learns underlying patterns
Very low training error	Low testing error
High model complexity	Balanced model complexity
Poor performance on unseen data	Good performance on unseen data

How to Improve Generalization

To prevent overfitting when using more hidden neurons:

- **Regularization (L1/L2):** Penalizes large weights and reduces model complexity.
- **Dropout:** Randomly deactivates neurons during training to prevent co-adaptation.
- **Early Stopping:** Stops training when validation performance stops improving.
- **Cross-Validation:** Helps estimate how well the model generalizes.
- **Increase Training Data:** More data allows the network to learn robust patterns.

Summary

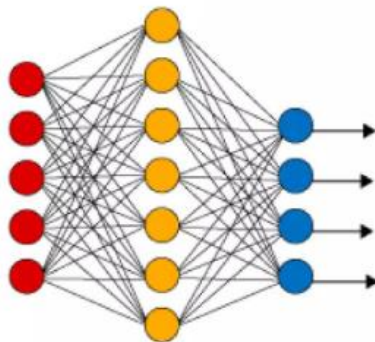
Hidden Neurons	Effect
Few neurons	Simpler model, may underfit complex data
Moderate neurons	Good balance between learning and generalization
Too many neurons	High representational power but greater risk of overfitting

Conclusion

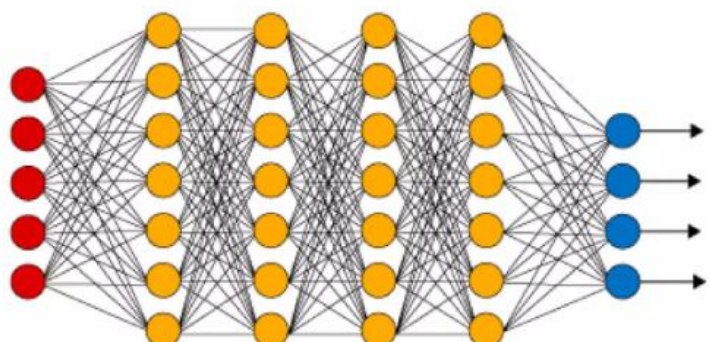
Adding more hidden neurons to an MLP increases its **representational power** by enabling it to learn more complex nonlinear relationships and approximate a wider range of functions. However, increasing the number of neurons also raises the risk of **overfitting**, leading to poor **generalization** on unseen data. Therefore, the best-performing MLP is typically one with an appropriate number of hidden neurons, combined with techniques such as **regularization, dropout, early stopping, and cross-validation** to achieve a balance between model complexity and predictive performance.

Combing Neurons in Hidden Layers: The “Emergent” Power to Approximate

Simple Neural Network



Deep Learning Neural Network



● Input Layer ● Hidden Layer ● Output Layer

Universality: For any arbitrary function $f(x)$, there exists a neural network that closely approximate it for any input x

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

Introduction

A **Deep Neural Network (DNN)** is trained by minimizing a loss function using **Stochastic Gradient Descent (SGD)** and **backpropagation**.

The SGD weight update rule is

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

where:

- w_t = current weights
- η = learning rate
- $\nabla L(w_t)$ = gradient of the loss function

Backpropagation computes these gradients layer by layer so that the weights can be updated to minimize the loss.

Curse of Dimensionality

The **curse of dimensionality** occurs when the number of features becomes very large.

As dimensionality increases:

- Data become sparse.
- Distance measures become less meaningful.
- More parameters must be learned.
- More training data are required.
- The model becomes more prone to overfitting.

Impact on Gradient Updates in Backpropagation

During backpropagation, gradients are computed for every weight.

In high-dimensional datasets:

- The number of weights increases significantly.
- Gradient calculations become more computationally expensive.

- Some gradients may become extremely small (**vanishing gradients**) or very large (**exploding gradients**).
- Updates become noisy because SGD uses only a small batch of samples.

Effect

- Slow and unstable learning.
- Difficulty reaching the global minimum.
- Increased memory and computation requirements.

Impact on Convergence Speed of SGD

SGD updates weights after processing each mini-batch.

In high-dimensional spaces:

- The optimization landscape becomes more complex.
- Many local minima, saddle points, and flat regions may exist.
- Gradient directions fluctuate considerably.
- More iterations are required to converge.

Consequences

- Slower convergence.
- Larger computational cost.
- Greater sensitivity to learning-rate selection.

Impact on Model Generalization

Generalization is the model's ability to perform well on unseen data.

With many features:

- The model may memorize the training data.
- Noise is learned instead of meaningful patterns.
- Training accuracy becomes very high.
- Test accuracy decreases.

This leads to **overfitting**.

Analytical Summary

Aspect	Effect of Curse of Dimensionality
Gradient Updates	Noisy, unstable, computationally expensive
SGD Convergence	Slower convergence and more optimization difficulty
Model Generalization	Higher risk of overfitting and poor test performance

Techniques to Overcome These Issues

1. Dimensionality Reduction (PCA)

Principal Component Analysis (PCA):

- Removes redundant and correlated features.
- Projects data into a lower-dimensional space.
- Preserves most of the important information.

Example:

10,000 Features

↓

500 Principal Components

Justification

- Fewer parameters to optimize.
- Faster gradient computation.
- Reduced computational cost.
- Lower risk of overfitting.
- Improved convergence of SGD.

2. Regularization (L1/L2)

Regularization modifies the loss function by adding a penalty term.

L2 Regularization:

$$L_{new} = L + \lambda \sum w^2$$

L1 Regularization:

$$L_{new} = L + \lambda \sum |w|$$

Justification

- Prevents excessively large weights.
- Encourages simpler models.
- Reduces variance.
- Improves generalization.
- Helps prevent overfitting.

Additional Techniques

3. Dropout

- Randomly disables neurons during training.
- Prevents neurons from relying too heavily on one another.
- Improves robustness and generalization.

4. Batch Normalization

- Normalizes activations within each mini-batch.
- Stabilizes gradient flow.
- Allows higher learning rates.
- Accelerates convergence.

5. Early Stopping

- Monitors validation loss during training.
- Stops training before the model begins to overfit.
- Improves performance on unseen data.

How These Techniques Improve Training

Technique	Benefit
PCA	Reduces dimensionality and computational complexity
Regularization	Controls model complexity and reduces overfitting
Dropout	Improves generalization by preventing neuron co-adaptation
Batch Normalization	Stabilizes gradients and speeds up convergence
Early Stopping	Prevents excessive fitting to training data

Conclusion

The **curse of dimensionality** negatively affects deep neural network training by making **backpropagation gradients noisier**, **slowing the convergence of SGD**, and **increasing the risk of overfitting**, which reduces model generalization. These issues can be effectively addressed through techniques such as **dimensionality reduction (PCA)** and **regularization (L1/L2)**, along with **dropout**, **batch normalization**, and **early stopping**. Together, these methods reduce model complexity, improve optimization efficiency, stabilize training, and enable the network to generalize better to unseen data.

